```
#%% md
# Plan de Recuperación ante Desastres (DRP) y Estrategia de Respaldo

## 1. Introducción y Alcance

Este documento describe el Plan de Recuperación ante Desastres (DRP) y la estrategia de respaldo para el **Sistema de Pronóstico de Demanda Collins**, basado en la base de datos SQLite (`Collins_Demand_DB.db`) y los scripts de Python asociados.

**Objetivo:** Garantizar la continuidad de las operaciones del sistema de pronóstico de demanda, minimizando el tiempo de inactividad (RTO) y la pérdida de datos (RPO) en caso de un desastre o fallo grave.

## 2. Evaluación de Riesgos

Los principales riesgos identificados para este sistema son:

| Riesgo | Impacto Potencial | Probabilidad |
| :--- | :--- | :--- |
| **Fallo del Servidor/Hardware** | Pérdida total de la aplicación y la base de datos. | Media |
| **Corrupción de la Base de Datos** | Datos de ventas históricas o pronósticos inutilizables. | Media |
| **Error Humano/Borrado Accidental** | Pérdida de archivos de código o datos específicos. | Baja |

## 3. Estrategia de Respaldo (Backup)

La estrategia de respaldo se centra en la simplicidad y la integridad de los datos, dado que el sistema utiliza un único archivo de base de datos SQLite.

### 3.1. Componentes a Respaldar
```

1. **Base de Datos:** Archivo `Collins_Demand_DB.db`.
2. **Código Fuente:** Todos los archivos `.py` y `.ipynb` (e.g., `run.py`, `app_dashboard.py`, `db_manager.py`, `SOLUTION_CHALLENGE_ENSAMBLE.ipynb`).
3. **Archivos de Datos Críticos:** `DatosHistóricosdePedidosydeFacturación.csv` (fuente de datos) y `Pronostico_Demanda_Top10x5_ENSEMBLE_FINAL.csv` (resultados clave).

3.2. Frecuencia y Retención

Componente	Frecuencia	Tipo de Respaldo	Retención
Base de Datos (`.db`)	Diaria (Fuera de horario laboral)	Completo	7 días
Código Fuente (`.py`, `.ipynb`)	Semanal / Tras cada cambio	Completo	30 días
Archivos de Datos (`.csv`)	Mensual	Completo	90 días

3.3. Ubicación del Respaldo

Se recomienda seguir la regla **3-2-1**:
*   **3** copias de los datos.
*   En **2** tipos de medios diferentes.
*   **1** copia fuera del sitio (off-site).

Ubicaciones Recomendadas:
1. **Local:** Disco duro secundario en el servidor principal.
2. **Nube (Off-site):** Almacenamiento en la nube (ej. Amazon S3, Google Cloud Storage, Azure Blob Storage) con cifrado.

4. Plan de Recuperación ante Desastres (DRP)

El DRP detalla los pasos para restaurar el servicio en un

```
 nuevo entorno (servidor de recuperación) tras un
desastre.

### 4.1. Fases de Recuperación

| Fase | Descripción | RTO Estimado |
| :--- | :--- | :--- |
| **1. Evaluación y Declaración** | Confirmar el desastre
 y declarar el inicio del DRP. | 1 hora |
| **2. Configuración del Entorno** | Provisionar un nuevo
 servidor y configurar el sistema operativo y
dependencias (Python, librerías). | 2 horas |
| **3. Restauración de Datos y Código** | Descargar el
último respaldo de la nube y restaurar los archivos. | 1
hora |
| **4. Verificación y Puesta en Marcha** | Ejecutar
pruebas de integridad de datos y lanzar la aplicación. |
1 hora |

**Tiempo Objetivo de Recuperación (RTO):** 5 horas.
**Punto Objetivo de Recuperación (RPO):** 24 horas (
basado en el respaldo diario).

### 4.2. Procedimiento Detallado de Recuperación

1.  **Provisionar Nuevo Servidor:** Instalar el sistema
operativo (ej. Ubuntu/Linux) y el entorno Python.
2.  **Instalar Dependencias:** Instalar todas las
librerías necesarias (ej. `pandas`, `sqlite3`, `streamlit
` si se usa para el dashboard).
```bash
# Ejemplo de instalación de dependencias
pip install -r requirements.txt # Asumiendo que existe un
 archivo de requisitos
```
3. **Restaurar Código Fuente:** Descargar y colocar todos
 los archivos de código en el directorio de trabajo.
4. **Restaurar Base de Datos:** Descargar el último
```

```
archivo `Collins_Demand_DB.db` del almacenamiento en la
nube y colocarlo en el directorio correcto.
5.  **Verificación de Integridad:**
*   Verificar que el archivo `.db` no esté corrupto.
*   Ejecutar el script de gestión de base de datos (`
db_manager.py`) para una prueba de conexión.
6.  **Lanzamiento de la Aplicación:** Iniciar la
aplicación principal (ej. `python run.py` o `streamlit
run app_dashboard.py`).
7.  **Pruebas de Aceptación del Usuario (UAT):** Confirmar
 que el dashboard muestra los datos correctos y que el
proceso de pronóstico se ejecuta sin errores.

## 5. Roles y Responsabilidades

| Rol | Responsabilidad Principal |
| :--- | :--- |
| **Administrador de Sistemas** | Mantenimiento de la
infraestructura, ejecución de la Fase 2. |
| **Desarrollador/Analista de Datos** | Verificación del
código, pruebas de integridad de datos, ejecución de la
Fase 4. |
| **Dueño del Negocio** | Declaración del desastre,
aprobación de la puesta en marcha. |

---

#%%
```